12 Le strutture di controllo: selezione

La struttura di **selezione** consente di far eseguire particolari istruzioni in alternativa ad altre sulla base del risultato di una condizione. Se la condizione è vera viene eseguito un blocco di istruzioni, altrimenti ne viene eseguito un altro. I **blocchi** sono indicati con le parentesi graffe.

In Java la selezione è realizzata con il seguente costrutto:

```
if (condizione)
{
   // istruzioni eseguite se la condizione è vera
}
else
{
   // istruzioni eseguite se la condizione è falsa
}
```

Il blocco indicato da *else* può non essere presente.
Se il blocco è formato da una sola istruzione, si può evitare di mettere le parentesi graffe. Comunque l'uso delle parentesi, anche in presenza di una sola istruzione, serve per ottenere una maggiore leggibilità ed evitare possibili errori di interpretazione.
La condizione che segue l'*if* può essere una variabile di tipo *boolean* oppure un'espressione che contiene gli operatori di confronto oppure gli operatori booleani.

• Operatori di confronto
L'operatore di uguaglianza è rappresentato da due simboli di uguale (==). Si faccia attenzione alla differenza rispetto all'operatore di assegnamento =, che usa un solo segno di uguale.
Per esempio, con la seguente struttura di selezione, sul video viene scritto il messaggio *sufficiente* se il voto è uguale a 6.

```
if (voto == 6)
{
   System.out.println("sufficiente");
}
```

La disuguaglianza (*diverso da*) è espressa usando l'operatore **!=**.
Gli altri operatori di confronto sono **<**, **<=**, **>**, **>=**.

• Operatori booleani
Gli operatori booleani vengono applicati a due valori booleani e restituiscono il risultato in base alle regole dell'algebra booleana.
L'operatore **&&** (oppure &) indica l'operazione di AND.
L'operatore **||** (oppure |) indica l'operazione di OR.
La negazione NOT viene espressa con l'operatore **!**.

I due operatori di AND e i due di OR forniscono gli stessi risultati. La differenza si nota in termini di efficienza.
L'espressione (*cond1 && cond2*) è equivalente a (*cond1 & cond2*): sono vere solo se entrambe *cond1* e *cond2* sono vere, mentre basta che una condizione sia falsa per rendere falsa l'espressione.
L'uso di *&&* è più efficiente perché, durante la valutazione, se *cond1* è falsa, *cond2* non viene valutata, perché comunque l'espressione è sicuramente falsa. Al contrario l'operatore & valuta sempre entrambe le condizioni. È quindi utile usare come operatori booleani *&&* e ||.

La struttura di **selezione multipla** consente di guidare l'esecuzione del programma scegliendo tra diverse alternative, a seconda del valore di una certa espressione.
L'espressione deve restituire un tipo intero oppure un carattere.
La struttura generale della selezione multipla in Java è la seguente:

```
switch (espressione)
{
   case valore1: // istruzioni
            break;
   case valore2: // istruzioni
            break;
   . . . .
   default: // istruzioni
            break;
}
```

L'espressione dopo la parola chiave **switch** solitamente è una variabile intera. All'interno del blocco si elencano i possibili valori che può assumere l'espressione, usando per ogni valore un blocco **case**.
Durante l'esecuzione, a seconda del valore dell'espressione, viene eseguito il blocco *case* associato. Se non viene trovato alcun valore nell'elenco dei *case*, si eseguono le istruzioni inserite dopo **default**. La parola chiave **break** serve per indicare la fine del blocco di istruzioni e fa terminare la selezione multipla.
Nel seguente frammento di programma viene mostrato come funziona il costrutto *switch* e come sia anche possibile unire tra loro più casi, associando ad essi un unico blocco di istruzioni.

```
int numero = (int) (Math.random() * 10);

System.out.println("E' stato estratto ");
switch (numero)
{
   case 1: System.out.println("il numero 1");
            break;
   case 0:
   case 2:
   case 4:
   case 6:
   case 8: System.out.println("un numero pari");
            break;

   default: System.out.println("un numero dispari diverso da 1");
            break;
}
```

Operazione di *casting*: converte il numero casuale in un intero, troncando la parte decimale.

Metodo **random** della classe **Math**: restituisce un numero casuale di tipo *double* compreso tra 0.0 e 1.0. Moltiplicando il numero restituito per 10 si ottiene un numero compreso tra 0.0 e 10.0.

La classe **Math** contiene i metodi per eseguire le operazioni matematiche.

13 Le strutture di controllo: ripetizione

La struttura di **ripetizione** (o *ciclo* oppure *iterazione*, in inglese *loop*) consente di eseguire un blocco di istruzioni più volte. In Java ci sono tre modalità per rappresentare questa struttura di controllo.

• while

La prima struttura di controllo iterativa assume la seguente forma:

```
while (condizione)
{
   // istruzioni
}
```

Le istruzioni contenute nel blocco vengono eseguite mentre la condizione si mantiene vera. La condizione viene controllata prima di entrare nel ciclo, quindi se è falsa, le istruzioni non vengono eseguite nemmeno una volta.

```
int a=2;
while (a <= 1024)
{
   System.out.println(a);
   a *= 2;
}
```

La variabile *a* viene raddoppiata mentre si mantiene minore o uguale di 1024.

Restituisce le potenze del numero *a* (2, 4, 8, 16, ...) fino a 1024.

• do while

La seconda struttura di controllo iterativa consente di effettuare il test sulla condizione alla fine del ciclo. In questo modo il blocco di istruzioni viene eseguito almeno una volta.
La struttura è la seguente:

```
do
{
   // istruzioni
}
while (condizione);
```

Le istruzioni vengono eseguite mentre la *condizione* si mantiene vera.

L'ultima riga deve terminare con un punto e virgola.

• for

La terza struttura di controllo iterativa consente di eseguire la ripetizione con un contatore. Inoltre raggruppa all'inizio del ciclo le fasi di inizializzazione, aggiornamento e controllo della condizione. La struttura generale è la seguente:

```
for (inizializzazione; condizione; aggiornamento)
{
   // istruzioni
}
```

Dopo la parola chiave *for* vanno inseriti, tra parentesi tonde, alcuni elementi separati dal punto e virgola.
La parte *inizializzazione* contiene il codice che viene eseguito una sola volta prima di entrare nel ciclo *for*. Solitamente contiene la dichiarazione di una variabile locale e la rispettiva inizializzazione.
La parte *condizione* contiene un'espressione booleana che viene valutata prima di eseguire il blocco di istruzioni. Se è falsa, non verranno mai eseguite le istruzioni presenti nel blocco. Se è vera, viene eseguito il ciclo.

Al termine del ciclo, vengono eseguite le istruzioni presenti nella parte *aggiornamento* e successivamente viene rivalutata la condizione per stabilire se la ripetizione deve continuare.

Per esempio, la seguente struttura *for* visualizza i primi 10 numeri naturali:

```
for (int i=1; i<=10; i++)
{
  System.out.println(i);
}
```

La variabile *i*, essendo stata dichiarata all'interno del *for*, è visibile solo nel blocco.

ESERCIZIO GUIDATO

Dati come input dieci voti di uno studente espressi con numeri, segnalare con un giudizio non numerico se il voto è negativo, sufficiente o positivo.

Algoritmo in pseudocodifica

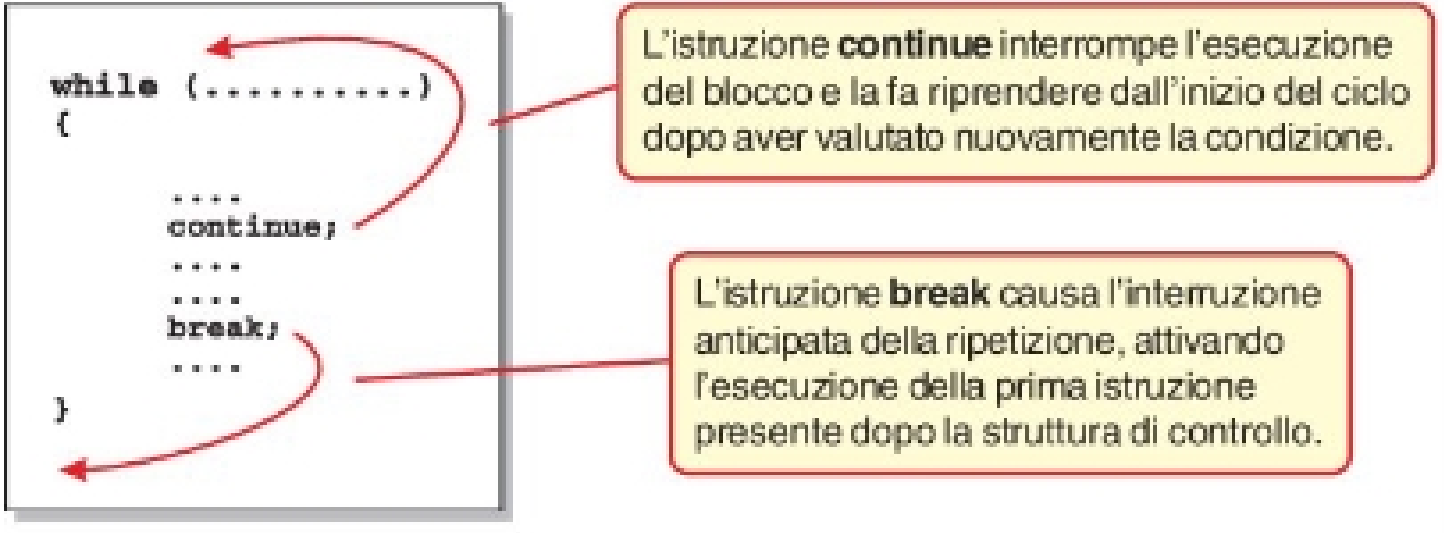
```
inizio
  per i da 0 a numeroVoti − 1
    immetti voto
    se voto < 6
    allora
        assegna testo = "negativo"
    altrimenti
        se voto == 6
        allora
            assegna testo = "sufficiente"
        altrimenti
            assegna testo = "positivo"
        fine se
    fine se
    scrivi testo
  ripeti
fine
```

Ripetizione con contatore.

Struttura di selezione annidata

Cicli interrotti e cicli infiniti

Le istruzioni presenti nei blocchi delle strutture iterative vengono eseguite in sequenza. È però possibile modificare l'ordine di esecuzione usando due istruzioni: *break* e *continue*.



Un programma Java può comunque essere interrotto in qualsiasi momento usando la combinazione di tasti **Ctrl + C**.
In particolare può capitare di scrivere un programma con strutture iterative che diventano dei *loop* infiniti, nei casi in cui la condizione del ciclo si mantiene sempre vera.

14 La struttura di dati array

Quando si devono memorizzare più oggetti che sono dello stesso tipo, si possono utilizzare gli **array** a una dimensione (*vettori*) o a più dimensioni (*matrici*).
Un array è realizzato con un riferimento che punta all'area di memoria dove si trovano i suoi elementi.
Per indicare l'array, cioè un gruppo di elementi dello stesso tipo, si usa un unico nome collettivo.
Ogni elemento è individuato attraverso un indice (numero intero).
Il primo elemento dell'array ha come indice 0. Il numero di elementi che compongono un array costituisce la *dimensione* dell'array. Quindi la posizione dell'ultimo elemento di un array è individuata dall'indice ottenuto decrementando la dimensione di 1.
Durante l'esecuzione del programma vengono controllati gli indici degli array e se non sono validi, cioè hanno superato i limiti consentiti, viene segnalato un errore.

Per poter creare e usare un array si devono seguire tre passaggi:
- dichiarazione dell'array
- allocazione
- eventuale inizializzazione.

La **dichiarazione di un array** consiste nella specificazione del suo tipo. Il tipo può essere un tipo predefinito di Java (per esempio *int, float*) oppure può essere una classe (per esempio *String, Automobile*). Si possono creare array che contengono un insieme di oggetti tutti della stessa classe.
Come per le variabili, la dichiarazione è fatta indicando un tipo e poi il nome da associare all'array. Però subito dopo il nome sono presenti le parentesi quadre, per indicare che è un array. Le parentesi potrebbero essere messe anche dopo il tipo, nel seguito si userà sempre la dichiarazione con le parentesi dopo il nome.

```
int i[];
String nomi[];
```

Array di interi.

Array di oggetti String.

La sola dichiarazione non specifica quale sia la dimensione dell'array. L'array, a questo punto, non è ancora stato creato e assume il valore speciale **null**.

L'**allocazione di un array** consente di specificare quanto spazio riservare per l'array, cioè quanti elementi saranno memorizzati al suo interno. L'allocazione viene eseguita utilizzando l'operatore **new**. È lo stesso operatore che sarà usato successivamente anche per allocare gli oggetti.
L'allocazione delle variabili dell'esempio precedente viene fatta con le seguenti istruzioni:

```
i = new int[5];
nomi = new String[10];
```

Array di 5 interi.

Array di 10 oggetti String.

L'operatore *new* è seguito dal tipo dell'array e dalla dimensione, specificata tra parentesi quadre. Dopo la fase di allocazione, si può dire che l'array è stato creato e può essere utilizzato, assegnando i valori ai suoi elementi.
È possibile riferirsi ai singoli elementi dell'array utilizzando il nome e un numero che indica la posizione all'interno dell'array. Il numero usato è l'**indice** dell'array.
Gli elementi dell'array *i* sono accessibili nel seguente modo:

i[0], i[1], i[2], i[3], i[4]

Come si vede, si parte con l'indice 0 per indicare il primo elemento e si termina con l'indice dato dalla dimensione meno 1, per indicare l'ultimo elemento. Essendo 5 la dimensione dell'array, l'ultimo elemento è rappresentato dall'indice 4.
Se si cerca di accedere a un elemento che non fa parte dell'array, per esempio *i[5]*, viene generata l'eccezione **ArrayIndexOutOfBoundsException** e il programma non viene eseguito correttamente.
Per assegnare i valori agli elementi di un array si procede così:

```
i[0] = 45;
i[1] = 12;
i[2] = 3;

nomi[0] = "Francesca";
```

Questi assegnamenti, se vengono eseguiti subito dopo la dichiarazione e l'allocazione, rappresentano l'**inizializzazione dell'array**.
Un array mantiene al suo interno l'informazione della sua dimensione. Per richiamare questa informazione si fa seguire al nome dell'array un punto e la parola **length**.
Per esempio:
i.length restituisce il numero 5
nomi.length restituisce 10.

Questo modo di indicare la dimensione dell'array è utile all'interno dei cicli ed evita di scrivere esplicitamente la dimensione attraverso un numero.
Per esempio, con il seguente ciclo *for*, si assegna a ogni elemento dell'array *i* il valore corrispondente al quadrato del suo indice.

```
for (int indice=0; indice < i.length; indice++)
{
  i[indice] = indice*indice;
}
```

Java consente di unire le due fasi di dichiarazione e di allocazione per rendere più veloce la creazione di array.

La **dichiarazione e allocazione degli array** possono essere fatte contemporaneamente sulla stessa riga:

```
int i[] = new int[5];
String nomi[] = new String[10];
```

È possibile anche dichiarare, allocare e allo stesso tempo inizializzare un array, inserendo tra parentesi graffe i valori che deve contenere.

```
int i[] = {45, 12, 3, 10, 2000};
```

La dimensione dell'array corrisponde al numero di elementi racchiusi tra parentesi graffe.

ESEMPIO 2

Il metodo main riceve come parametro un array di stringhe chiamato args[]. In questo array sono contenuti i parametri che vengono passati al programma dalla riga di comando. Scrivere un programma che stampa tutti i parametri che vengono passati.

Il programma deve controllare se ci sono parametri forniti dall'utente insieme al comando. In caso positivo, ne visualizza il valore con una ripetizione che va da 0 a *args.length* − 1, essendo *args.length* la dimensione dell'array *args[]*.

Programma Java (*Parametri.java*)

```
class Parametri
{
   public static void main(String args[])
   {
      System.out.println("Elenco parametri:");

      if (args.length == 0)
      {
         System.out.println("nessun parametro");
         return;
      }
      for(int i=0; i<args.length; i++)
      {
         System.out.println(args[i]);
      }
   }
}
```

Se si esegue il programma da linea comandi con i seguenti tre parametri:

```
java Parametri 45 -t Rossi
```

si ottiene come risultato:

```
Elenco parametri:
45
-t
Rossi
```

15 Gli array multidimensionali

Gli array considerati finora corrispondono in matematica ai *vettori*.
Esistono altri tipi di array chiamati *multidimensionali*, in particolare l'array bidimensionale, che corrisponde in termini matematici a una **matrice** con righe e colonne. Ogni elemento della matrice è individuato da una coppia di numeri di cui il primo indica la riga e il secondo la colonna.
Lo schema seguente presenta una matrice di dimensione 3x4, all'interno di ogni cella sono indicati gli indici di riga e di colonna.

0,0	0,1	0,2	0,3
1,0	1,1	1,2	1,3
2,0	2,1	2,2	2,3

In Java la dichiarazione e l'allocazione di un array a due dimensioni procede allo stesso modo della dichiarazione di un array monodimensionale: occorre specificare la nuova dimensione usando un secondo paio di parentesi quadre.

```
int matrice[][];
matrice = new int[3][4];
```

Per l'assegnamento di valori alla matrice bisogna riferirsi a uno specifico elemento della matrice indicando la sua posizione con una coppia di indici.

```
matrice[0][0] = 10;
```

Assegna il valore 10 al primo elemento della matrice.